

A Trace-Language Theory of Agents: Goal-Driven Type-0 Producers Bounded by Data-Driven Type-3 Consumers

Scott Weeden
Ms of Computer Applications
Texas Tech University
Lubbock, Texas, USA
sweeden@ttu.edu

Akbar Siami Namin
Dr of Computer Science
Texas Tech University
Lubbock, Texas, USA
akbar.namin@ttu.edu
Faculty page

Abstract

Koohestani, Li, Podkopaev, and Izadi recently asked, “Are agents just automata?” [1]. We take that question as motivation for a narrower one: *what can be proved about agents if they are observed only through the finite traces of operations they emit?* This paper develops a trace-language account of agent behaviour and applies it to a producer–consumer architecture in which a goal-directed generator proposes traces and a data-driven deterministic verifier accepts only traces satisfying a regular specification. The verifier is implemented as an Aho–Corasick keyword automaton [2] compiled from unit-test, schema, and submission-validity checks.

The central formal point is deliberately modest. If an agent’s trace language is intersected with a regular verifier language, the result remains in the original Chomsky class. Consequently, conformance to the verifier specification can be checked in linear time by the DFA, even when membership in the generator’s full trace language remains undecidable. We therefore distinguish *trace validation* from full language membership: the verifier establishes $\tau \in L(\mathcal{V})$ for an emitted trace τ , not a complete decision procedure for $\tau \in L(\mathcal{G}) \cap L(\mathcal{V})$ when $\mathcal{G}$ is modelled by an unrestricted generator.

We instantiate the framework on the SPACESHIP TITANIC task from MLE-BENCH. The start artifact `data/trace_language_original.csv` specifies twelve seed sub-agent alphabets; the run artifact `data/trace_language_kaggle.csv` records one accepted trace with thirteen columns, including an emergent `kaggle_trainer` column. These artifacts are treated as empirical traces, not as proofs of exact Chomsky-class membership. Where the paper assigns Type-2, indexed, Type-1, or Type-0 abstractions to practical sub-agents, those assignments are modelling hypotheses unless accompanied by an explicit recogniser or grammar.

The resulting system produced a leaderboard placement of 710 out of 2330 teams ($\approx 30.5\%$, top tertile). This experiment shows that language-constrained trace validation can guide a practical agentic workflow; it does not by itself validate the stronger formal classification hypotheses. Reproducible code is published at <https://www.kaggle.com/sweeden-ttu/competitions/spaceship-titanic>.

Index Terms

Agentic systems, trace language, Chomsky hierarchy, deterministic finite automata, Aho–Corasick, recursive enumeration, producer–consumer, trace validation, language-constrained validation, ML engineering.

I. INTRODUCTION

THE contemporary literature on autonomous software agents is rich in architectural diagrams and short on formal definitions. An agent is variously described as “a system that perceives and acts”, “a planner that calls tools”, or “an LLM in a loop”, and each definition is stated at a level of abstraction that admits neither a decision procedure nor a complexity bound. The consequence is that two agents with radically different computational power are routinely compared as if they belonged to a single class, and verification claims about agentic systems are correspondingly difficult to falsify.

We are not the first to ask whether the Chomsky hierarchy is the right scaffold for thinking about this problem. In particular, Koohestani, Li, Podkopaev, and Izadi pose the question *Are*

agents just automata? [1] and answer affirmatively, establishing a memory-based equivalence between simple-reflex agents and finite automata, hierarchical task-decomposition agents and pushdown automata, and unrestricted-memory agents and Turing machines [3]. We accept their answer. Their question, however, is the preliminary one. The question that does practical work is operational: *given that agents are automata, how can the Chomsky type of one agent be used to bound the Chomsky type of another?* This is the producer–consumer problem of Dijkstra [4] re-stated in the language of formal grammars. The present paper offers a methodology that answers it in the affirmative, with software verification and validation techniques inferred into the consumer side and the idealised theoretical apparatus of automata built into the producer side.

A. Why a narrow definition

We deliberately keep the definition of an agent narrow, taking only what is forced upon us by the theory of automata Turing introduced in 1936 [3] and the language hierarchy Chomsky introduced in 1956–59 [5], [6]. An agent, for us, is an effectful machine that, when started on an admissible input, emits a finite string of operation symbols and then halts (or fails to halt). The collection of all such strings, taken across the input domain, is the agent’s *trace language*. Two agents with the same trace language are, by stipulation, the same agent. Whatever cannot be read off from the trace language is, for the purposes of this paper, not treated as an agent property.

The motivation for keeping the definition narrow is twofold. First, it gives a direct line from the agent to the Chomsky hierarchy and to the recognising automata. An agent whose trace language is regular (Type-3) is realisable by a finite-state controller and admits decidable verification in linear time [7]. An agent whose trace language is context-free (Type-2) is realisable by a pushdown controller and admits $O(n^3)$ membership [8]. An agent whose trace language is recursively enumerable (Type-0) is, in the worst case, *undecidable to verify* — a fact whose modern statement we inherit from Robinson’s resolution of Hilbert’s tenth problem [9], [10], which established that recursively enumerable sets are exactly the Diophantine sets and that the membership problem for them admits no general algorithm.

Second, the narrow definition gives an empirical handle. If the agent is studied through its traces, then those traces can be recorded, stored, projected, and checked against explicit language specifications. A finite experiment produces finite strings, not a proof of the full generating language, so we use empirical traces as validation artifacts and as evidence for modelling hypotheses. This places agent trace validation near the V&V of compilers, parsers, and protocol implementations [11], [12], while preserving the distinction between observed executions and language-class theorems.

B. The Goal-Driven producer and Data-Driven consumer

Our central architectural claim is the producer–consumer restatement. There is exactly one *Goal-Driven* producer agent, modelled by an idealised recursive-enumeration envelope. In practice it is realised as a large language model wrapped in a tool-calling loop, and the loop is permitted to enumerate finite-length sequences of tool invocations from the union of the sub-agents’ alphabets. The Goal-Driven producer has no a priori knowledge of *what* the goal is; it knows only the specification of the input domain and the tool catalogue.

There is exactly one *Data-Driven* consumer agent, the *verifier*. Its trace language is Type-3: regular and, by Rabin and Scott’s construction [7], decidable in linear time. The verifier is concretely realised as the deterministic finite automaton of an Aho–Corasick keyword index [2] compiled from the competition’s empirical fixtures: the unit-test corpus, the submission-validity tests, the column-name checks, and the integrity tests on the

`train.csv/test.csv/sample_submission.csv` triple.

The binding is the entirety of our architectural claim:

A single idealised Goal-Driven producer, constrained by a regular Data-Driven verifier, can yield a validated trace.

Type-2 and Type-1 sub-agents are not designed; they are *derived*. They appear as the canonical projections of the recursively enumerated trace onto the parts of the orchestration substrate that admit, respectively, a stack discipline (the back-edges of the planner/developer/reviewer loop, the recursive `retry_phase_on_low_score`) and a linear-bounded workspace (the deterministic Aho–Corasick scanner and the LBA-style `compute_dependency_graph` that walks the phase-to-phase dependency relation). We make these projections precise in Section V.

A subtlety: nothing in the framework prevents a Type-3 agent from itself acting as a producer, in which case it backward-infers constrained changes onto the tape (e.g., updates to the keyword set K as new task fixtures arrive). Such a Type-3 producer is admissible *provided* the inference terminates in a decidable verdict. When it does not, the classical halting problem of Turing [3] re-asserts itself on the Type-3 producer’s loop and the architecture loses its verifier-side budget. We discuss the engineering consequence in Section X: in the realised system, decidability is enforced by the regular envelope of the verifier and, where the data-driven side begins to drift, by a final human verification and validation step that closes the loop.

C. The start language and its recursive enumeration

The system is initialised with a single *start language* L_0 , given concretely in `data/trace_language_original.csv`. The start language contains twelve sub-agents and their seed action alphabets — nothing else. The Goal-Driven producer, constrained by the Data-Driven Type-3 verifier, then enumerates a finite trace for the SPACESHIP TITANIC task instance. The result is the observed trace artifact L_1 , recorded in `data/trace_language_kaggle.csv`. L_1 contains *thirteen* sub-agents. The thirteenth, `kaggle_trainer`, is the emergent agent: it does not exist in L_0 and is materialised *by the act of enumeration*, after the verifier first refuses traces that omit the training step.

We use this enumeration as the operational analogue of recursive enumeration in the agentic setting. Turing’s machines enumerate strings; our idealised producer enumerates tool-call strings; and the connection between the emitted strings and the empirical Kaggle competition is mediated by the Aho–Corasick verifier.

D. Idealisation and physical bounding

It is worth being clear about the difference between the theory and the machine that actually ran the experiment. The Goal-Driven agent’s trace language is *idealised* Type-0. The actual execution was bounded by the finite tape and finite

RAM of a single laptop and, for the heavier ablation runs, by the High Performance Computing Center at Texas Tech University. The realised machine is, formally, a (very large) linear-bounded automaton — Type-1. The idealisation is sound for our purposes: every trace producible by the realised LBA is also producible by the idealised Turing machine, and conformance to the verifier language is, by the DFA construction in Theorem 2, checked in linear time. The *physical* computation is therefore far more restricted than the theoretical envelope, and any trace emitted by the bounded machine is contained in the idealised envelope used for analysis. This is the standard idealisation of Hartmanis and Stearns [13], and we adopt it on those terms.

E. Contributions

- 1) A formal definition of the trace language $L(\mathcal{A})$ of an agent $\mathcal{A}$ as the language generated by an effectful controller over a finite operation alphabet, together with a proof that the resulting object satisfies the Chomsky hierarchy verbatim (Section IV).
- 2) A producer–consumer answer to the Koohestani–Li–Podkopaev–Izadi question: a single idealised Goal-Driven producer, intersected with a single Data-Driven Type-3 consumer realised as an Aho–Corasick DFA, suffices to produce a validated trace while preserving the producer class and keeping verifier-side conformance checking linear (Section V, Theorem 2).
- 3) A demonstration that Type-2-style and Type-1-style roles can be modelled as projections of the observed trace onto stack-discipline and bounded-workspace substrates respectively (Section VII).
- 4) A discussion of how a Type-3 agent may itself act as a producer, backward-inferring constrained changes onto the tape, under the necessary condition that decidability be preserved (Section X); failing this condition, the halting problem returns and the verifier-side budget collapses.
- 5) An empirical instantiation on the SPACESHIP TITANIC task from MLE-BENCH achieving placement 710/2330 ($\approx 30.5\%$, top tertile). The start language `trace_language_original.csv`, the recursively enumerated trace `trace_language_kaggle.csv`, and the notebooks that produced the submission are released at <https://www.kaggle.com/sweeden-ttu/competitions/spaceship-titanic> (Sections VIII–IX).

F. Roadmap

Section II surveys related work, opening with Koohestani et al. and the producer–consumer framing. Section III fixes notation drawn from Turing’s automata theory and Chomsky’s hierarchy. Section IV gives the rigorous definition of the trace language. Section V classifies trace languages by Chomsky type and proves the closure theorem. Section VI situates Aho’s indexed grammars and the Aho–Corasick verifier in

the framework. Section VII describes the Goal-Driven/ Data-Driven binding and the recursive enumeration of the start language. Sections VIII–IX report the experimental study and result on SPACESHIP TITANIC. Section X discusses idealisation, physical bounding, and the Type-3-as-producer subtlety. Section XI concludes.

II. RELATED WORK

The literature relevant to this paper falls into five strands. The first asks whether agents are automata, and answers affirmatively in a memory-architecture sense (Koohestani et al. [1]). The second is the producer–consumer framing of cooperating sequential processes (Dijkstra [4]), which is the operational form of the question we actually wish to answer: how can the type of one agent bound another? The third is the classical theory of automata and the Chomsky hierarchy (Turing, Chomsky, Rabin and Scott, Hopcroft and Ullman, Aho). The fourth is software verification and validation (Hoare, Dijkstra, Pnueli, Clarke and Emerson, Lamport). The fifth is the boundary between decidable and undecidable verification fixed by Robinson and the MRDP theorem. We address each in turn.

A. Are agents just automata?

Koohestani, Li, Podkopaev, and Izadi [1] recently posed and answered the title question, establishing a memory-architecture equivalence: simple-reflex agents are finite automata; hierarchical task-decomposition agents are pushdown automata; agents employing read/write memory for reflection are Turing machines. Their paper introduces a *right-sizing principle*: choose the minimal computational class sufficient for the task.

Two observations are owed to that paper. First, the equivalence they establish is the architectural premise of the present work; without it, the question of how Chomsky classes interact across agents would have no canonical formulation. Second, their paper is preliminary in scope: it classifies agents but does not prescribe a construction in which one agent’s class *constrains* another’s class. We supply that construction here. Where they ask *Are agents just automata?* and answer *yes*, we ask *can the type of one agent bound another?* and answer *yes, with V&V techniques inferred into a Data-Driven verifier and the idealised theoretical apparatus of automata built into a Goal-Driven producer*.

A related strand, Formal-LLM [14], encodes plan-construction constraints as a context-free grammar and guides an LLM-based planner with a pushdown automaton. Our construction is dual: we do not constrain plan generation through a higher-class grammar, we *verify* plan emission through a lower-class one. Aho–Corasick is the “less-Turing-complete-than-the-LLM” device that earns us the verifier-side budget of Theorem 2.

B. The producer–consumer framing

Dijkstra’s cooperating sequential processes [4] fix the producer–consumer formulation: one process produces a

sequence of artefacts; another consumes them; correct co-operation requires that the consumer never reads an artefact the producer has not yet produced, and the producer never overwrites an artefact the consumer has not yet consumed. The classical question is: *what synchronisation discipline is sufficient?* In our setting the question becomes: *what Chomsky-class discipline on the consumer is sufficient to render the producer’s recursively enumerable output decidable in finite time?* The answer, by Theorem 2, is Type-3. Lower would not work (Type-3 is the minimal class that contains the keyword matchers Aho–Corasick implements); higher would work but raise the verifier-side budget unnecessarily and risk the halting problem returning through the consumer’s loop.

C. Automata theory and the Chomsky hierarchy

Turing’s 1936 paper [3] introduces the abstract machine that bears his name and proves the existence of a universal one. Every formal claim in this paper inherits, at some depth, from that construction. Chomsky’s 1956 and 1959 papers [5], [6] fix the hierarchy itself: regular, context-free, context-sensitive, and unrestricted (Type-3 through Type-0). Rabin and Scott [7] fix the formal correspondence between regular languages and finite automata, with the nondeterminism that makes the proofs tractable. Hopcroft, Motwani, and Ullman [8] consolidate the modern treatment with strict inclusions and standard parsing complexities. Hartmanis and Stearns [13] introduce the time and space complexity classes in which our Type-1/Type-0 distinction ultimately lives, and supply the formal vocabulary for our “idealised Type-0 versus realised LBA” discussion in Section X.

Aho’s 1968 paper [15] introduces indexed grammars as a strict extension of context-free grammars, positioned strictly between Type-2 and Type-1. In our framework the back-edge structure of an iterative repair loop — e.g., the `retry_phase_on_low_score` cycle in Section VII — inhabits exactly that zone. We use Aho’s indexed grammars as the bridge between the pushdown sub-agent (Type-2) and the linear-bounded sub-agent (Type-1).

The Aho–Corasick paper [2] compiles a finite keyword set into a deterministic finite automaton that recognises the union of the keywords in linear time and constant space. In our framework Aho–Corasick is *the* canonical realisation of the Data-Driven Type-3 consumer.

D. Software verification and validation

Hoare’s axiomatic basis [12] and Dijkstra’s guarded-command calculus [16] supply the machinery of Hoare triples $\{P\} C \{Q\}$ that we apply to each unit-test step in the Aho–Corasick keyword set. The predicates P and Q are the “before” and “after” states of a single tool invocation in the trace.

Pnueli’s 1977 paper [17] introduces the linear temporal logic in which we phrase the safety and liveness properties of the Goal-Driven loop:

$$G(plan \rightarrow F \text{ verified})$$

expresses that every plan emitted by the producer is eventually accepted or refuted by the consumer. Clarke and Emerson [18] introduce the branching variant we use to reason about the back-edges of the iterative repair loop, and Clarke, Grumberg, and Peled [19] consolidate the model-checking framework that, in principle, decides the Type-3 component of our verifier.

Lamport [20], [21] supplies the ordering on events in the multi-agent execution and the temporal logic of actions; the typesetting of this paper itself uses his $\text{\LaTeX}$ system [22]. Liskov’s abstract data types [23] are the type discipline underlying the `TypedDict` state objects that flow between sub-agents.

E. The undecidability boundary: Robinson and MRDP

The boundary between decidable and undecidable verification — which, in our framework, is the boundary between Type-1 and Type-0 agents — is fixed by Hilbert’s tenth problem and its resolution. Robinson’s 1949 paper [9] and the Davis–Putnam–Robinson follow-up [10] established that the recursively enumerable subsets of $\mathbb{N}^k$ are exactly the Diophantine subsets, and that the membership problem for an arbitrary recursively enumerable set admits no general algorithm. Our idealised Goal-Driven producer is analysed against this recursive-enumeration backdrop, but the empirical verifier does not decide full producer membership. It decides the narrower and operationally useful question of whether a candidate trace conforms to the regular verifier specification.

F. Complexity, learning, and symbolic AI

Cook [24] and Karp [25] fix the NP-completeness frame within which the model-search step of the Goal-Driven producer (the Optuna tuning of LightGBM, the cross-validated stacking ensemble) is, formally, an instance of constrained combinatorial optimisation. Yao [26] supplies the notion of expected complexity used when reporting average trace lengths over the input distribution. Blum [27] fixes the machine-independent measure in which we report the validated trace language’s complexity.

The supervised-learning component of the Spaceship Titanic solution is justified by the deep-learning manifesto of LeCun, Bengio, and Hinton [28]; the causal discipline that prevents the verifier from being fooled by spurious correlations is justified by Pearl [29]. McCarthy [30] is the symbolic representation of programs as data — the property that lets a Goal-Driven agent emit a *program* (a Kaggle notebook) as the terminal element of its trace. Newell and Simon [31] state the physical-symbol-system hypothesis under which the agent’s tool calls are the elementary symbols. We adopt their hypothesis as a working assumption rather than as a contribution.

G. Synthesis

The strands above suffice to support every formal claim in the sequel. In particular, our novelty relative to Koohestani et al. [1] is the producer–consumer construction: where they classify, we bound; where they say which class an agent inhabits, we say which class *must* consume an agent of class

i in order for the system as a whole to remain decidable. The empirical instantiation in Sections VIII–IX is reported as data, not as theory, and stands or falls on its own reproducibility artefacts.

III. PRELIMINARIES: TURING’S THEORY OF AUTOMATA

We fix notation drawn directly from Turing [3], Rabin and Scott [7], and Hopcroft, Motwani, and Ullman [8]. Nothing in this section is original. The section exists because every claim in the sequel is stated in terms of these objects, and any deviation from their classical definitions would propagate as ambiguity in our results.

A. Alphabets, strings, and languages

A finite, non-empty set Σ is an *alphabet*. A finite sequence $w = a_1 a_2 \dots a_n$ with $a_i \in \Sigma$ is a *string*; its length is $|w| = n$; the empty string is ε with $|\varepsilon| = 0$. The set of all finite strings over Σ is Σ^* , and $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$. A *language* over Σ is a subset $L \subseteq \Sigma^*$. Concatenation is associative with identity ε . We write $L_1 L_2 = \{wv : w \in L_1, v \in L_2\}$ and $L^* = \bigcup_{n \geq 0} L^n$.

B. Grammars and the Chomsky hierarchy

A *grammar* is a tuple $G = (V, \Sigma, P, S)$ where V is a finite set of nonterminals, Σ is the (terminal) alphabet disjoint from V , $P \subseteq (V \cup \Sigma)^* V (V \cup \Sigma)^* \times (V \cup \Sigma)^*$ is a finite set of production rules, and $S \in V$ is the start symbol. The language $L(G) \subseteq \Sigma^*$ is the set of terminal strings derivable from S by repeated application of rules in P .

The Chomsky hierarchy stratifies grammars by the form of the rules in P :

- Type-0 (unrestricted): $\alpha, \beta \in (V \cup \Sigma)^*$ with α containing at least one nonterminal. $L(G)$ is *recursively enumerable*.
- Type-1 (context-sensitive): $|\beta| \geq 1$, plus optionally $S \rightarrow \varepsilon$. $L(G)$ is *context-sensitive*.
- Type-2 (context-free): $A \in V$. $L(G)$ is *context-free*.
- Type-3 (regular): $A \rightarrow a$ with $A, B \in V$, $a \in \Sigma$ (right-linear, equivalently left-linear). $L(G)$ is *regular*.

The strict inclusions

$$\mathcal{L}_3 \subsetneq \mathcal{L}_2 \subsetneq \mathcal{L}_1 \subsetneq \mathcal{L}_0$$

hold by elementary construction [8].

C. Recognising automata

To each Chomsky class corresponds a class of recognising machines.

- Finite Automaton (FA, (CQ, Σ , Q , q_0 , F), Type-3) with Q finite, $\delta : Q \times \Sigma \rightarrow Q$ deterministic (or $Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$ nondeterministic), $q_0 \in Q$, $F \subseteq Q$. Membership decides in $O(|w|)$ time by Rabin and Scott [7].
- Pushdown Automaton (PDA, (PQ, Σ , Γ , q_0 , F), Type-2) with Q finite, Γ a finite LIFO stack. Membership decides in $O(|w|^3)$ time by the standard CYK or Earley parser [11].

Linear-Bounded Automaton (LBA, (LQ, Σ , Q , q_0 , F), Type-1) head is restricted to the cells initially occupied by the input. Membership is PSPACE-complete in $|w|$ [8].

Turing Machine (TM, (CQ, Σ , Q , q_0 , q_{rej}), Type-0) with Γ the tape alphabet ($\Sigma \subseteq \Gamma$), $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ a partial function, and the standard two-way tape semantics of Turing [3]. Membership is, in general, *undecidable*; the universal machine of Turing [3] establishes that the recursively enumerable languages are exactly the languages generated by Type-0 grammars.

D. Recursive enumeration and undecidability

A language $L \subseteq \Sigma^*$ is *recursively enumerable* (r.e., or *semi-decidable*) if there exists a Turing machine M such that, started on the empty input, M prints exactly the strings of L on its output tape (in some order, possibly with repetitions). Equivalently, L is r.e. iff there exists a Turing machine M_L that halts and accepts on input w iff $w \in L$, and may run forever on $w \notin L$.

A language L is *decidable* (*recursive*) if both L and its complement $\bar{L}$ are r.e.; equivalently, if there exists a Turing machine that halts on every input and accepts iff $w \in L$. The strict inclusion of decidable r.e. languages inside r.e. languages is a theorem of Turing [3]; the undecidability of the membership problem for arbitrary r.e. languages is the *Halting Problem*, also Turing’s.

A celebrated and non-elementary characterisation of the r.e. class is Robinson’s: the r.e. subsets of $\mathbb{N}^k$ are exactly the Diophantine subsets [9], [10]. The corollary — that there is no algorithm deciding membership in an arbitrary r.e. set defined by Diophantine constraints — is the form in which we use the result. An idealised Type-0 agent therefore inherits, by the same line of argument, the property that no algorithm decides whether a given trace is in the agent’s trace language.

E. Idealised versus physically bounded machines

Every physical computation is bounded: the tape is a finite array of cells, the time is finite, the energy is finite. The *idealisation* we adopt is the standard one of Hartmanis and Stearns [13]: we analyse machines whose tape and time are unbounded in the formal specification, while observing that any concrete execution witnesses only a finite prefix of the idealised computation.

For a Turing machine M , fix a tape bound $b \in \mathbb{N}$ and a time bound $t \in \mathbb{N}$. The bounded machine $M[b, t]$ is the restriction of M to executions in which the tape head visits at most b cells in at most t steps. Plainly $M[b, t]$ is a finite state machine: its state space is bounded by $|Q| \cdot |\Gamma|^b \cdot b \cdot (t + 1)$. Therefore:

Lemma 1 (Bounded soundness). *If M accepts w in $\leq t$ steps using $\leq b$ tape cells, then $M[b, t]$ accepts w . Equivalently, every trace producible by the bounded machine $M[b, t]$ is producible by the idealised machine M .*

Lemma 1 is the formal warrant for the “idealised Type-0” framing in Section VII. The Goal-Driven agent, as implemented on the author’s laptop, is a very large but finite LBA.

For the analysis we treat it as an idealised TM; any trace the bounded machine emits is also emitted by the idealised machine, and the verification claims we discharge against the idealised machine apply a fortiori to the bounded one.

F. Closure properties

We will need the following standard results [8]:

- 1) Regular languages are closed under union, intersection, complement, concatenation, and Kleene star.
- 2) Context-free languages are closed under union, concatenation, and Kleene star, but *not* under intersection or complement; however, the intersection of a context-free language with a regular language is context-free.
- 3) Context-sensitive languages are closed under union, intersection, and concatenation; closure under complement was established later (Immerman–Szelepcsényi, beyond our citation discipline; we use only union and intersection in this paper).
- 4) Recursively enumerable languages are closed under union, intersection, and concatenation; intersection with a regular language is r.e.

The closure of r.e. under intersection with regular languages is the closure that drives Theorem 2: the validated trace language — the intersection of the Goal-Driven Type-0 agent’s trace language with the Data-Driven Type-3 verifier’s trace language — is r.e., but its membership question is *decided* by the verifier’s DFA whenever a trace is emitted to the disk. Conformance checking against the verifier specification remains Type-3.

IV. THE TRACE LANGUAGE OF AN AGENT

This section is the formal core of the paper. We define an agent as a tuple of seven objects, three of which are derivative. We then define the agent’s trace language and prove that the definition satisfies the Chomsky hierarchy verbatim. The construction is austere and self-contained; nothing beyond Section III is assumed.

A. Operation alphabet

Definition 1 (Operation alphabet). An *operation alphabet* is a finite, non-empty set Σ whose elements $\sigma \in \Sigma$ each denote an effectful computation *name*, called an *operation symbol*. Each σ is associated with a (possibly partial) computable function $\llbracket \sigma \rrbracket : X \rightarrow Y$ acting on a state space X and producing a successor state in $Y \subseteq X$, so that the abstract symbol carries an effect when executed. The function $\llbracket \cdot \rrbracket$ is fixed once and for all by the runtime environment and is not a property of the agent.

For the empirical instantiation in Section VIII, Σ is the union of the column entries of `trace_language_kaggle.csv`. Examples include `load_vector_db(gemini_embeddings.json)`, `compose_planner_prompt`, `invoke_gemini(gemini-2.5-flash-lite)`, `run_phase_unit_tests(phase; comp_dir)`, and

`train_stacked_ensemble(LGBM+XGB+HistGB+linear_meta)`. The operation symbol is the *name plus its parenthesised argument list as written in the source*; two invocations of the same name with different arguments are different operation symbols, exactly as Liskov’s abstract data types prescribe for parameterised operations [23].

B. Agent

Definition 2 (Agent). An *agent* is a 7-tuple

$$\mathcal{A} = (\Sigma_{\mathcal{A}}, Q_{\mathcal{A}}, q_0^{\mathcal{A}}, X_{\mathcal{A}}, \delta_{\mathcal{A}}, F_{\mathcal{A}}, h_{\mathcal{A}})$$

where

- $\Sigma_{\mathcal{A}} \subseteq \Sigma$ is the agent’s operation alphabet;
- $Q_{\mathcal{A}}$ is a (possibly countably infinite) *configuration space*;
- $q_0^{\mathcal{A}} \in Q_{\mathcal{A}}$ is the *initial configuration*;
- $X_{\mathcal{A}}$ is the (computable) *input domain*;
- $\delta_{\mathcal{A}} : Q_{\mathcal{A}} \times X_{\mathcal{A}} \rightarrow Q_{\mathcal{A}} \times \Sigma_{\mathcal{A}}^*$ is the agent’s *step relation*: at configuration q on input x it returns a successor configuration q' together with a finite string of operation symbols emitted by the step;
- $F_{\mathcal{A}} \subseteq Q_{\mathcal{A}}$ is the *halting set*;
- $h_{\mathcal{A}} : F_{\mathcal{A}} \rightarrow \{\text{accept}, \text{reject}\}$ is the *halting verdict*.

The objects $X_{\mathcal{A}}, \delta_{\mathcal{A}}, h_{\mathcal{A}}$ are derivative: $X_{\mathcal{A}}$ is determined by Σ and the runtime environment; $\delta_{\mathcal{A}}$ is the agent’s *program*; and $h_{\mathcal{A}}$ is fixed by the convention that a configuration in $F_{\mathcal{A}}$ accepts iff it carries the post-condition for which the trace was emitted.

Remark 1. The 7-tuple is the minimal one that lets us speak of an emitted *string* per step. A 5-tuple of the $(Q, \Sigma, \delta, q_0, F)$ Hopcroft–Ullman [8] form would force δ to emit a single symbol per step, which is too restrictive: a single “call to the planner” in our empirical agent emits a structured prompt-build sequence that we prefer to record as a string of three or four operation symbols. The choice of $\Sigma_{\mathcal{A}}^*$ rather than $\Sigma_{\mathcal{A}}$ in the codomain of $\delta_{\mathcal{A}}$ is therefore essential.

C. Run, trace, and trace language

Definition 3 (Run). A *run* of $\mathcal{A}$ on input $x \in X_{\mathcal{A}}$ is a finite or infinite sequence

$$\rho = ((q_0, w_0), (q_1, w_1), (q_2, w_2), \dots)$$

with $q_0 = q_0^{\mathcal{A}}$, $w_0 = \varepsilon$, and for each $i \geq 1$, $\delta_{\mathcal{A}}(q_{i-1}, x) = (q_i, w_i)$ defined. The run is *halting* if it is finite and its last configuration q_n lies in $F_{\mathcal{A}}$; it is *accepting* if additionally $h_{\mathcal{A}}(q_n) = \text{accept}$.

Definition 4 (Trace). The *trace* of a run $\rho = ((q_0, w_0), \dots, (q_n, w_n))$ is the concatenation

$$\tau(\rho) = w_0 \cdot w_1 \cdots w_n \in \Sigma_{\mathcal{A}}^*.$$

We write $\tau(\mathcal{A}, x)$ for the trace of the run of $\mathcal{A}$ on x when that run is unique.

Definition 5 (Trace language). The *trace language* of $\mathcal{A}$ is

$$L(\mathcal{A}) = \{ \tau(\rho) : \rho \text{ is an accepting run of } \mathcal{A} \text{ on some } x \in X_{\mathcal{A}} \} \subseteq \Sigma_{\mathcal{A}}^*.$$

D. Trace-equivalence

Definition 6 (Trace equivalence). Two agents $\mathcal{A}_1, \mathcal{A}_2$ over a common alphabet $\Sigma_{\mathcal{A}}$ are *trace-equivalent*, written $\mathcal{A}_1 \equiv_{\tau} \mathcal{A}_2$, iff $L(\mathcal{A}_1) = L(\mathcal{A}_2)$.

The austerity claim of the paper, in two lines, is: *An agent is its trace language, and only its trace language.* Trace-equivalent agents are the same agent. The internal architecture (LLM, neural network, hand-coded controller, mixture of all three) is irrelevant to the identity criterion.

E. Multi-agent systems

Definition 7 (Multi-agent system). A *multi-agent system* is a finite tuple $\mathcal{M} = (\mathcal{A}_1, \dots, \mathcal{A}_k, R)$ where each $\mathcal{A}_i$ is an agent and $R \subseteq \Sigma^* \times \{1, \dots, k\} \times \{1, \dots, k\}$ is an *orchestration relation* specifying which agent's emission, on which prefix, may be consumed by which other agent.

The trace language of $\mathcal{M}$ is the set of strings over the common alphabet $\Sigma = \bigcup_i \Sigma_{\mathcal{A}_i}$ that are obtainable by the orchestrated shuffle: pick an accepting run of each $\mathcal{A}_i$, then interleave their traces in any order consistent with R . Formally,

$$L(\mathcal{M}) = \bigcup_{(\tau_1, \dots, \tau_k) \in L(\mathcal{A}_1) \times \dots \times L(\mathcal{A}_k)} \text{shuffle}_R(\tau_1, \dots, \tau_k)$$

where shuffle_R is the standard interleaving restricted by R [8].

F. The CSV files are languages

We make explicit the empirical correspondence used throughout the sequel. The header row of `trace_language_kaggle.csv` names the sub-agents:

$\mathcal{A}_1 = \text{research_agent}, \dots, \mathcal{A}_{13} = \text{kaggle_trainer}$.

For $1 \leq i \leq 13$, the i -th column of the file (skipping empty cells) is a single string $\tau_i \in \Sigma_{\mathcal{A}_i}^*$. This string is one accepting run of $\mathcal{A}_i$ on the SPACESHIP TITANIC input. The trace language $L(\mathcal{A}_i)$ contains, in principle, infinitely many other strings; the column records exactly the one that the recursive enumeration emitted.

The same correspondence holds for `trace_language_original.csv`, with the difference that its header has only twelve columns (the start language has no `kaggle_trainer` agent) and its row entries are substantially shorter.

G. Closure under intersection with the verifier

We now state the standard closure result that is the architectural pivot of the paper. Its role is deliberately modest: it justifies using a regular verifier as a conformance filter over traces emitted by a more expressive producer. It does not make full membership in the producer language decidable.

Theorem 2 (Regular conformance closure). *Let $\mathcal{G}$ be an agent with $L(\mathcal{G})$ in Chomsky class $\mathcal{L}_i$ for some $i \in \{0, 1, 2, 3\}$. Let $\mathcal{V}$ be an agent with $L(\mathcal{V}) \in \mathcal{L}_3$ (a regular language). Define the validated composition $\mathcal{G} \parallel \mathcal{V}$ as the trace filter whose accepted*

traces are exactly those emitted by $\mathcal{G}$ and accepted by $\mathcal{V}$'s DFA. Then

$$L(\mathcal{G} \parallel \mathcal{V}) = L(\mathcal{G}) \cap L(\mathcal{V}) \in \mathcal{L}_i.$$

In particular, regular conformance filtering does not raise the Chomsky class of the producer language.

Proof. Each Chomsky class $\mathcal{L}_i$ is closed under intersection with regular languages. For $i = 3$, this is the standard product-construction [7]. For $i = 2$, it is the standard product of a PDA with a DFA, which is again a PDA [8]. For $i = 1$, it follows from LBA closure under intersection with regular languages. For $i = 0$, it is the standard product of a Turing machine with a DFA, which is again a Turing machine [3], [8]. $\square$ $\square$

Corollary 3 (Verifier-side budget). *Given a candidate trace τ of length n , checking verifier conformance $\tau \in L(\mathcal{V})$ runs in $O(n)$ time and $O(1)$ space. This is a verifier-side statement only: full membership in $L(\mathcal{G}) \cap L(\mathcal{V})$ still requires membership in $L(\mathcal{G})$, which may be undecidable when the producer is Type-0.*

H. Empirical traces and language-class hypotheses

The empirical CSV files record finite traces, not complete trace languages. A single observed trace cannot determine the Chomsky class of the language that generated it: the same finite string is consistent with a finite regular language, with Σ^* , and with many non-regular languages. We therefore use `trace_language_original.csv` and `trace_language_kaggle.csv` as artifacts for trace validation and hypothesis generation, not as proofs of strict language-class membership.

In the implementation, the Goal-Driven component is best read as an *idealised Type-0 envelope*: the design permits recursive enumeration of operation strings, while the concrete laptop execution is necessarily finite and hence bounded. Lemma 1 justifies comparing bounded executions with an idealised envelope, but it does not prove that the physical artifact realizes a language in $\mathcal{L}_0 \setminus \mathcal{L}_1$. The defensible empirical claim is narrower: the observed trace conforms to the regular verifier specification, and the resulting artifacts illustrate how language-theoretic validation can be applied to agent traces.

V. CHOMSKY CLASSIFICATION OF AGENTS

We now classify trace languages by Chomsky type and exhibit, for each type, the canonical agent shape from the empirical SPACESHIP TITANIC system. The classification is the literal one of Section III; there is no new mathematics here, only the application of an old hierarchy to a new object. The novelty, if any, is the identification of which sub-agents in the empirical CSVs inhabit which class.

A. Type-3: regular trace languages

Definition 8 (Type-3 agent). An agent $\mathcal{A}$ is *Type-3* if $L(\mathcal{A}) \in \mathcal{L}_3$, equivalently, if there exists a finite automaton M with $L(M) = L(\mathcal{A})$.

In the empirical system, the Type-3 agents are `tools_index` and `unit_test_runner`. Both have trace languages over a small finite alphabet whose strings are generated by a finite-state controller:

- `tools_index`: the trace language is the regular expression

`(load_schema)·(compose_tool_text)·(embed_tool_text)`

read straight off the column. No back-edge appears. No stack discipline is required.

- `unit_test_runner`: the trace is a sequence of `test_*` keywords drawn from the finite test catalogue. The deciding finite automaton is the Aho–Corasick DFA over the catalogue [2].

The Data-Driven verifier $\mathcal{V}$ of this paper is realised as the union of these two columns, compiled into a single Aho–Corasick DFA. Membership in $L(\mathcal{V})$ is decided in $O(n)$ time and $O(1)$ space per candidate trace, by Rabin and Scott’s classical construction [2], [7].

B. Type-2: context-free trace languages

Definition 9 (Type-2 agent). An agent $\mathcal{A}$ is *Type-2* if $L(\mathcal{A}) \in \mathcal{L}_2 \setminus \mathcal{L}_3$, equivalently, if there exists a pushdown automaton M with $L(M) = L(\mathcal{A})$ and $L(\mathcal{A})$ is not regular.

In the empirical system, the Type-2 sub-agents are `planner_agent` and `reviewer_agent`. Both exhibit nested structure that is naturally modelled with a stack. The planner emits a step-list whose nesting depth is determined by the EDA bundle:

```
emit_step(id; title; instructions; tools)
  -> emit_step(...) -> ...
emit_success_criteria
emit_plan_json
```

and the reviewer’s score-and-clamp loop pushes a configuration onto an implicit stack each time a sub-test fails:

```
collect_failed_tests -> compose_reviewer_prompt
  -> clamp_score_to_2_on_unit_failure
  -> clamp_score_to_1_5_range
  -> emit_review_result(...).
```

The stack discipline is implicit but recoverable: the trace language of `planner_agent` matches *Dyck-language shape* [8] when one identifies opening `compose_planner_prompt` and closing `emit_plan_json` as bracket symbols.

C. The indexed-grammar zone (between Type-2 and Type-1)

Aho’s indexed grammars [15] occupy a class $\mathcal{L}_{\text{indexed}}$ strictly between $\mathcal{L}_2$ and $\mathcal{L}_1$. Their distinguishing feature is a finite *stack of indices* attached to each nonterminal during a derivation.

In the empirical system, the indexed-zone sub-agent is `sop_orchestrator`, whose trace language exhibits a finite-index repair stack: the tokens `cap_iterations_at_max(=3)` and

`retry_phase_on_low_score` together form an indexed loop whose nesting is bounded by the constant 3. This loop resembles the bounded-index bookkeeping that indexed grammars capture. We do not prove non-context-freeness here; we use the indexed zone as a modelling abstraction for the coupled iteration and phase counters.

D. Type-1: context-sensitive trace languages

Definition 10 (Type-1 agent). An agent $\mathcal{A}$ is *Type-1* if $L(\mathcal{A}) \in \mathcal{L}_1 \setminus \mathcal{L}_{\text{indexed}}$, equivalently, if there exists a linear-bounded automaton M with $L(M) = L(\mathcal{A})$ and $L(\mathcal{A})$ is not in the indexed class.

The Type-1-style sub-agents are `task_orchestrator` and `kernel_pipeline`. Their trace languages exhibit linear-bounded workspace growth: each step’s emitted string is bounded by a constant times the input length, and the `compute_dependency_graph(prev_stage_to_next_stage)` operation is, by construction, an LBA-style traversal of the phase graph that visits each cell at most a constant number of times.

The `kernel_pipeline`’s trace contains `generate_kernel_packages(4_copies)`, `write_packages_to_disk`, and `sequential_push_on_vm(stage_order)` — a strictly linear workspace expansion in the number of stages. No string in $L(\text{kernel_pipeline})$ exceeds a constant multiple of the input description’s length, which is the LBA hallmark.

E. Type-0: recursively enumerable trace languages

Definition 11 (Type-0 agent). An agent $\mathcal{A}$ is *Type-0* if $L(\mathcal{A}) \in \mathcal{L}_0 \setminus \mathcal{L}_1$, equivalently, if there exists a Turing machine M with $L(M) = L(\mathcal{A})$ and $L(\mathcal{A})$ is not context-sensitive.

The components for which we use an idealised Type-0 envelope are `mlebench_solver`, the `developer_agent` (in its idealised code-generation mode), and the `kaggle_trainer`. The `kaggle_trainer` column is not a proof of Type-0 membership; it is an empirical witness that the implementation used a recursively structured enumeration of training operations `bootstrap_indices`, `fit_model_on_subset`, `score_train_and_val`, `accumulate_oof_predictions`, and `track_best_episode_by_val_score`, parameterised by loop bounds (`for_each_fold_for_each_episode`) that are themselves outputs of the agent’s own deliberation.

The *Goal-Driven* agent of the architecture is therefore modelled by an idealised recursive-enumeration envelope under the orchestration relation R given in Section VII. The concrete implementation remains bounded; undecidability enters only as a property of the idealised envelope, not as an empirical claim about the finite run.

F. Summary table

G. The Goal-Driven / Data-Driven binding, formally

The architecture of the paper is now fully formal. Let

$$\mathcal{G} = \mathcal{A}_{\text{research}} \parallel \mathcal{A}_{\text{developer}} \parallel \mathcal{A}_{\text{mlebench}} \parallel \mathcal{A}_{\text{trainer}}$$

TABLE I

LANGUAGE-CLASS ABSTRACTIONS SUGGESTED BY THE EMPIRICAL SUB-AGENT TRACES. STRICT CLASS MEMBERSHIP IS ASSERTED ONLY WHERE AN EXPLICIT RECOGNISER IS SUPPLIED; THE REMAINING ROWS ARE MODELLING HYPOTHESES.

Sub-agent	Abstraction	Evidence
tools_index	Type-3	FA / DFA
unit_test_runner	Type-3	FA / Aho–Corasick DFA
reader_agent	Type-3	FA
summarizer_agent	Type-3	FA
planner_agent	Type-2?	PDA hypothesis
reviewer_agent	Type-2?	PDA hypothesis
sop_orchestrator	Indexed?	Indexed-grammar hypothesis (Aho [15])
task_orchestrator	Type-1?	LBA hypothesis
kernel_pipeline	Type-1?	LBA hypothesis
research_agent	Type-0?	TM hypothesis
developer_agent	Type-0?	TM hypothesis
mlebench_solver	Type-0?	TM hypothesis
kaggle_trainer	Type-0?	TM hypothesis (emergent)

under the orchestration relation R_G that allows any of these sub-agents to consume the prefix emitted by any of the others. Then $\mathcal{G}$ is modelled by an idealised recursive-enumeration envelope.

Let

$$\mathcal{V} = \mathcal{A}_{\text{tools_index}} \parallel \mathcal{A}_{\text{unit_test_runner}}$$

under the orchestration relation R_V that requires every `test_*` emission to be matched against the `tools_index` catalogue. Then $L(\mathcal{V}) \in \mathcal{L}_3$ for the verifier specification.

The validated trace language is

$$L(\mathcal{G} \parallel \mathcal{V}) = L(\mathcal{G}) \cap L(\mathcal{V})$$

which, by Theorem 2, remains in the generator’s model class, while conformance to $L(\mathcal{V})$ is decidable in $O(n)$ on the verifier side. The projected Type-2-style sub-agents `planner_agent`, `reviewer_agent` and the indexed-zone sub-agent `sop_orchestrator` and the Type-1-style sub-agents `task_orchestrator`, `kernel_pipeline` are not part of $\mathcal{G}$ or $\mathcal{V}$ as designed; they are the *projections* of $\mathcal{G} \parallel \mathcal{V}$ onto the orchestration graph, and they appear because the binding forces them into existence — not because the architect designed them.

The minimality claim is therefore:

One idealised goal-driven generator and one concrete Type-3 Data-Driven verifier suffice for the trace-validation architecture. Everything else is a projected modelling view.

This is the operational meaning of “a single Goal-Driven agent tamed by the Data-Driven verifier results in success.”

VI. INDEXED GRAMMARS AND THE AHO–CORASICK VERIFIER

This section deepens the role played by Aho’s two great contributions to the theory of formal languages: indexed grammars [15], which fix the language class strictly between context-free and context-sensitive, and the Aho–Corasick [2] multi-pattern matcher, which compiles a finite keyword set into a deterministic linear-time recogniser. The two papers were written eight years apart, but in our framework they play

complementary roles: indexed grammars provide a candidate abstraction for the iterative repair loop inside the Goal-Driven generator; the Aho–Corasick DFA is the realisation of the Data-Driven verifier.

A. Indexed grammars

An *indexed grammar* [15] is a 5-tuple

$$G = (V, \Sigma, F, P, S)$$

where V is the set of nonterminals, Σ is the alphabet of terminals, F is a finite set of *index symbols*, $S \in V$ is the start symbol, and P is a finite set of productions each of one of three forms:

- (a) $A[\xi] \rightarrow \alpha[\xi]$
- (b) $A[\xi] \rightarrow \alpha[f\xi]$ (push $f \in F$)
- (c) $A[f\xi] \rightarrow \alpha[\xi]$ (pop $f \in F$)

where $\alpha \in (V \cup \Sigma)^*$ and ξ is a string of indices attached to each nonterminal in α . The string attached to a nonterminal acts as a stack of indices that may be pushed at derivation steps of form (b), popped at form (c), and inherited unchanged at form (a).

Lemma 4 (Aho 1968). $\mathcal{L}_2 \subsetneq \mathcal{L}_{\text{indexed}} \subsetneq \mathcal{L}_1$.

The strict inclusion is witnessed by the language $\{a^n b^n c^n : n \geq 0\}$, which is in $\mathcal{L}_{\text{indexed}}$ but not in $\mathcal{L}_2$. The analogous pattern we look for empirically is the *paired iteration counter* pattern in `sop_orchestrator`’s trace, where the `cap_iterations_at_max(=3)` bound is enforced synchronously across the planner, developer, and reviewer sub-traces.

B. Indexed-grammar hypothesis for the iterative repair loop

In `sop_orchestrator`’s trace we observe the cycle

```

invoke_planner
-> invoke_developer
-> invoke_reviewer
-> compare_score_to_advance_threshold(>=3)
-> [ if score < 3:
    retry_phase_on_low_score (loop)
    cap_iterations_at_max(=3) ]
-> invoke_summarizer_on_advance
-> emit_outcome(complete|fail) .

```

Three observations motivate an indexed-grammar abstraction, while falling short of a separation proof:

- 1) The iteration counter is shared across three nested sub-traces (planner / developer / reviewer); the same counter appears synchronously in each. This is a cross-serial dependency, which would require proof before excluding a PDA.
- 2) The counter is bounded by a constant (3); this rules out arbitrary linear-bounded behaviour. The counter is not a tape, it is an index.
- 3) Each invocation of the loop pushes a fresh *index configuration* (the EDA bundle, the failed-test list, the clamped

score) that is consumed in inverse order during the `forward_artifact_to_next_phase` step.

These three properties are the ones Aho identified in 1968 as motivating the indexed-grammar extension [15]. We therefore treat `sop_orchestrator` as an indexed-zone hypothesis. Establishing strict membership would require an explicit indexed grammar and a non-context-freeness argument, which we leave as future formalisation.

C. Aho–Corasick as the Data-Driven verifier

The Aho–Corasick algorithm [2] takes a finite keyword set $K \subseteq \Sigma^*$ of total size $\|K\| = \sum_{w \in K} |w|$ and produces a deterministic finite automaton M_K such that, for any input string $x \in \Sigma^*$, M_K identifies all occurrences of all $w \in K$ in $O(|x| + \|K\| + z)$ time, where z is the number of matches. The construction has three steps:

- 1) Build a trie of K (the *goto* function).
- 2) Compute, for each trie node, a *failure* link to the longest proper suffix that is also a prefix of some keyword.
- 3) Compose the goto and failure into a single deterministic transition function δ_K .

In our setting Σ is the operation alphabet of Section IV, K is the set of operation symbols that appear in the unit-test catalogue and tools-index manifest, and the verifier $\mathcal{V}$ is the agent whose configuration space is the state set of M_K . Concretely K is read off the columns `tools_index` and `unit_test_runner` of `trace_language_kaggle.csv`; we list it in Appendix A.

Proposition 5 (Verifier as DFA). *The Data-Driven verifier $\mathcal{V}$ is realisable as a deterministic finite automaton over Σ with state set $Q_{\mathcal{V}}$ of size $O(\|K\|)$. For any candidate trace $\tau \in \Sigma^*$, the verdict $h_{\mathcal{V}}(\tau) \in \{\text{accept}, \text{reject}\}$ is computed in $O(|\tau|)$ time.*

Proof. Construct the Aho–Corasick DFA M_K over the keyword set K extracted from the two columns above; declare its accepting states to be those visited iff at least one keyword from each required category (e.g., `test_submission_validity`, `test_column_names_submission`) is matched. The size bound $O(\|K\|)$ is the standard size of an Aho–Corasick DFA, and the linear-time membership decision is the standard property of a DFA [2], [7]. $\square$ $\square$

D. Why Aho–Corasick rather than a generic LLM judge

A generic LLM-as-judge could, in principle, also play the role of $\mathcal{V}$. We deliberately chose Aho–Corasick for three reasons:

- 1) *Class containment.* An LLM-as-judge would not supply a transparent regular recogniser. An Aho–Corasick DFA is Type-3 and gives an $O(n)$ verifier-side budget by Corollary 3.
- 2) *Auditability.* The keyword set K is finite, human readable, and version-controlled; the failure links and goto table are derivable mechanically. An LLM-as-judge offers no such audit surface.

- 3) *Robustness against a generative producer.* If the generator is permitted to enumerate arbitrary strings, the verifier must be unable to be “talked into” accepting a malformed trace. A DFA cannot be talked into anything; it either matches or it does not. This is the precise sense in which the Goal-Driven producer is constrained by the Type-3 verifier.

E. The recursive enumeration step, decoded

We can now describe the recursive enumeration step that takes the start language $L_0 = L(\text{trace_language_original.csv})$ to the enumerated language $L_1 = L(\text{trace_language_kaggle.csv})$ in explicit terms.

- 1) Let Σ_0 be the operation alphabet of L_0 and K_0 be the corresponding keyword set extracted from its `tools_index` and `unit_test_runner` columns. $\mathcal{V}_0 = \mathcal{V}(K_0)$ is the corresponding Aho–Corasick DFA.
- 2) Run the Goal-Driven generator $\mathcal{G}_0$ over the SPACESHIP TITANIC input, intersecting every emitted prefix with $L(\mathcal{V}_0)$ and rejecting prefixes that exit the language of $\mathcal{V}_0$.
- 3) During execution, $\mathcal{G}_0$ proposes new operation symbols not in Σ_0 (e.g., the `wire_spaceship_feature_engineer(...)` symbol). Each new symbol is admitted to Σ_1 iff its emission satisfies $\mathcal{V}_0$ when treated as a keyword extension; the verifier’s keyword set is correspondingly extended to K_1 *only* for the duration of this enumeration.
- 4) The accumulated trace is written to disk as `trace_language_kaggle.csv`. Its alphabet Σ_1 properly extends Σ_0 ; its column count grows from twelve to thirteen by the addition of the `kaggle_trainer` column, which is the column of operations that were never in Σ_0 but became necessary under the verifier’s pressure.

The procedure is, in the operational sense, a bounded analogue of recursive enumeration: the agent enumerates strings, the verifier filters them, and the recursion terminates when the enumerated language satisfies all properties of the data-driven specification. In the literal sense of Robinson [9], [10] the recursion has no general termination guarantee; in our bounded laptop the recursion is forced to halt by the LBA bound described in the next section.

VII. METHODOLOGY: THE GOAL-DRIVEN / DATA-DRIVEN ARCHITECTURE

This section describes the architecture in operational terms. The architecture is austere by design: one idealised goal-driven generator, one Data-Driven Type-3 verifier, and an explicit recursive enumeration procedure that maps the start language to the realised trace language. The intermediate sub-agent classes are treated as projections and modelling hypotheses, not as independently proven language separations.

A. The two principal agents

a) *Goal-Driven generator $\mathcal{G}$* : $\mathcal{G}$ is modelled by an idealised unrestricted generator realised concretely as a bounded large-language-model loop over the union of the operation alphabets in `trace_language_original.csv`. Its configuration space is the LLM’s context window plus a finite set of bookkeeping flags; its step relation $\delta_{\mathcal{G}}$ samples a continuation, parses the continuation’s tool-call markup, and emits the corresponding operation string. The model used at run time is `invoke_gemini(gemini-2.5-pro)` for the open-ended deliberation steps and `invoke_gemini(gemini-2.5-flash-lite)` for the finite-state housekeeping steps.

$\mathcal{G}$ is given exactly the following information at start-up:

- the start-language alphabet Σ_0 (the operations columns of `trace_language_original.csv`);
- the empty workspace directory `/comp_dir/`;
- the boilerplate Kaggle CLI configuration.

$\mathcal{G}$ is *not* given the goal label (Transported), the public/private leaderboard specification, or any prior solution. Its idealised trace language is, in principle, a subset of Σ_0^* , restricted only by what $\mathcal{G}$ ’s LLM core happens to enumerate.

b) *Data-Driven verifier $\mathcal{V}$* : $\mathcal{V}$ is the Aho–Corasick DFA M_K described in Section VI and Proposition 5. Its keyword set K is the union of the operation symbols in the `unit_test_runner` and `tools_index` columns of `trace_language_original.csv`, extended only by the mechanical inclusion of the column-name and dtype tests generated from the SPACESHIP TITANIC `train.csv` schema. $\mathcal{V}$ has no LLM component; it is a deterministic linear-time checker. Its decision function is

$$h_{\mathcal{V}}(\tau) = \begin{cases} \text{accept} & \text{if every required keyword class is matched in } \tau \\ \text{reject} & \text{otherwise.} \end{cases}$$

B. The binding

The architecture’s single load-bearing claim is that

$$L(\mathcal{G} \parallel \mathcal{V}) = L(\mathcal{G}) \cap L(\mathcal{V}) \in \mathcal{L}_0$$

with conformance to $L(\mathcal{V})$ decidable in $O(n)$ on the verifier side (Theorem 2, Corollary 3).

In implementation, the binding is realised by an outer loop:

- 1) $\mathcal{G}$ proposes a candidate trace prefix τ_t .
- 2) $\mathcal{V}$ runs its DFA on τ_t , accepting or rejecting in linear time.
- 3) If $\mathcal{V}$ rejects, the rejected suffix is discarded and $\mathcal{G}$ is advised (by feeding back the failed-test list as a percept) to enumerate further.
- 4) If $\mathcal{V}$ accepts and the accepted trace contains a `write_submission_csv` step, the loop terminates and the submission is uploaded.

The loop terminates because the laptop’s tape is finite (see Section X), or because the verifier accepts a trace ending in `yield_final_result`, whichever comes first.

C. Projection of intermediate sub-agent views

The projected sub-agent views `planner_agent`, `reviewer_agent`, `task_orchestrator`, and `kernel_pipeline` were not designed as separate formal machines. They are projections of the constrained trace language $L(\mathcal{G} \parallel \mathcal{V})$ onto the orchestration substrate. Their Chomsky labels are modelling hypotheses unless an explicit recogniser or grammar is supplied.

Definition 12 (Projection of a trace language). Let $\mathcal{A}$ be an agent with operation alphabet $\Sigma_{\mathcal{A}}$ and trace language $L(\mathcal{A}) \subseteq \Sigma_{\mathcal{A}}^*$. Let $\Sigma' \subseteq \Sigma_{\mathcal{A}}$ be a sub-alphabet. The *projection* of $L(\mathcal{A})$ onto Σ' is the set

$$\pi_{\Sigma'}(L(\mathcal{A})) = \{ \pi_{\Sigma'}(\tau) : \tau \in L(\mathcal{A}) \}$$

where $\pi_{\Sigma'} : \Sigma_{\mathcal{A}}^* \rightarrow \Sigma'^*$ deletes every symbol not in Σ' .

Theorem 6 (Derived sub-agents). Let Σ_{planner} be the operation alphabet of the `planner_agent` column of `trace_language_kaggle.csv`. Then

$$L(\text{planner_agent}) = \pi_{\Sigma_{\text{planner}}}(L(\mathcal{G} \parallel \mathcal{V})).$$

The class of $L(\text{planner_agent})$ is no greater than $\mathcal{L}_0$ (because projection cannot raise class) and is shown by inspection in Section V to be $\mathcal{L}_2$. Analogous statements hold for `reviewer_agent` (Type-2), `sop_orchestrator` (indexed), `task_orchestrator` (Type-1), and `kernel_pipeline` (Type-1).

Sketch. The validated trace language is the language of all interleaved multi-agent emissions admissible under the orchestration relation. Projecting onto the planner’s alphabet selects exactly the planner’s emitted symbols. The class bound is the standard projection lemma [8]; the empirical class assignment is by structural inspection of the column. $\square$ $\square$

The minimality slogan of the paper is therefore literally true: the architect designs only $\mathcal{G}$ and $\mathcal{V}$. The intermediate-class agents emerge from the binding.

D. The recursive-enumeration procedure

Algorithm 1 is the procedure that maps the start language to the realised trace language.

The structure of Algorithm 1 is exactly the structure of an open-ended generator step (line 3) intersected with a Type-3 advance (line 4). By Theorem 2, the loop’s output language $L(\mathcal{G} \parallel \mathcal{V})$ is at most $\mathcal{L}_0$ and at least $\mathcal{L}_3$ (the verifier’s class). By the laptop’s finite tape, the loop terminates in finite time.

E. What the algorithm produces

When Algorithm 1 is run on the SPACESHIP TITANIC input, the operation alphabet Σ_0 extends to $\Sigma_1 = \Sigma_0 \cup \{\text{symbols emitted along the validated path}\}$. The thirteenth column of the resulting CSV is the `kaggle_trainer` column: a strictly new sub-agent whose alphabet contains symbols absent from Σ_0 (e.g., `train_stacked_ensemble` (LGBM+XGB+HistGB+linear_meta), `tune_lightgbm_with_optuna` (`n_trials=30`)),

Algorithm 1. RECURSIVELYENUMERATE($\mathcal{G}, \mathcal{V}, x$)

Require: Goal-Driven $\mathcal{G}$, Data-Driven verifier $\mathcal{V}$, input x .
Ensure: Verified trace $\tau \in L(\mathcal{G} \parallel \mathcal{V})$, or halt-on-tape-bound.

```
1:  $q \leftarrow q_0^{\mathcal{G}}; \tau \leftarrow \varepsilon; s \leftarrow q_0^{\mathcal{V}}.$ 
2: loop
3:    $(q', w) \leftarrow \delta_{\mathcal{G}}(q, x).$ 
4:    $s' \leftarrow \delta_{\mathcal{V}}^*(s, w)$  # advance the Aho–Corasick DFA on  $w$ .
5:   if  $s'$  is a reject state then
6:     feed the failed-keyword witness back to  $\mathcal{G}$  as a percept.
7:   else
8:      $\tau \leftarrow \tau \cdot w; q \leftarrow q'; s \leftarrow s'.$ 
9:     if  $q \in F_{\mathcal{G}}$  and  $s \in F_{\mathcal{V}}$  then
10:      return  $\tau.$ 
11:     end if
12:   end if
13:   if laptop tape exhausted then
14:     return halt-on-tape-bound.
15:   end if
16: end loop
```

Fig. 1. Recursive enumeration of the start language.

`bootstrap_indices(85pct; per_episode_random_state)`). Its emergence is forced by the verifier rejecting any trace that omits a training step, not by the architect having pre-designed it.

The empirical CSV is therefore a fixed point of the binding:

$$\Sigma_1 = \Sigma_0 \cup \text{Range}(\delta_{\mathcal{G}})|_{\mathcal{V}\text{-accepting}}$$

and the validated trace $\tau^* \in L(\mathcal{G} \parallel \mathcal{V})$ is exactly the column-wise concatenation of the recorded entries in `trace_language_kaggle.csv`.

F. Hoare-triple obligation per step

Each step of Algorithm 1 carries a Hoare-triple obligation in the sense of Hoare [12] and Dijkstra [16]:

$$\{P_t\} \quad w_t \quad \{P_{t+1}\}$$

where P_t is the precondition holding on the state space before the t -th emitted operation w_t , and P_{t+1} is the postcondition that the verifier checks via its $\delta_{\mathcal{V}}$ transition. When the precondition fails, the verifier rejects; when the postcondition fails, the verifier also rejects; the trace is admitted iff every Hoare triple along the emitted path holds.

This is the formal sense in which the architecture is *validation-by-construction*: the verifier’s DFA is the mechanical realisation of a finite Hoare-style proof obligation for the Goal-Driven generator’s emitted trace.

G. Temporal-logic statement of correctness

The correctness property of the binding is captured in LTL [17] by the safety-and-liveness pair

Safety: $G(\text{emit} \rightarrow \text{verifier} - \text{tracking})$

Liveness: $G(\text{plan} \rightarrow F(\text{verified} \vee \text{halt} - \text{on} - \text{tape} - \text{bound}))$.

Safety says that every emitted symbol advances the verifier’s state; liveness says that every plan eventually either succeeds verification or hits the tape bound. Both are decided by inspection of the model-checker on the orchestrator’s CFG [18], [19].

This concludes the formal description of the architecture. We turn next to the empirical instantiation.

VIII. EXPERIMENTAL SETUP

A. Benchmark

We instantiate the architecture on the SPACESHIP TITANIC task drawn from the publicly hosted MLE-BENCH suite (github.com/openai/mle-bench). SPACESHIP TITANIC is a binary-classification competition in which the agent must predict, for each test passenger, whether they were transported to an alternate dimension during a fictional interstellar collision. The competition is hosted on Kaggle as a “Getting Started” competition, has 2330 active teams as of the date of submission, and uses public-leaderboard accuracy as its ranking metric.

The task is selected for three reasons: it requires non-trivial feature engineering (compound Cabin field decomposition, group-aware k-fold splitting, missingness-indicator generation); it has a clearly defined submission validity criterion (column names, row count, no missing predictions); and its leaderboard is sufficiently dense that a 2–5% placement improvement constitutes a reproducible, statistically meaningful signal.

B. Hardware and software environment

- **Hardware:** a single laptop (the author’s macOS development machine). CPU and RAM bounds are recorded in the run log; both are finite and the realised computation is therefore an LBA in the formal sense of Section III.
- **Operating system:** macOS, with the Kaggle CLI version locked in the run-time configuration.
- **LLM backbone:** Google Gemini, accessed through the `google-genai` client. Two model strings appear in the realised trace: `gemini-2.5-pro` for Type-0 deliberation steps; `gemini-2.5-flash` and `gemini-2.5-flash-lite` for finite-state house-keeping and verification-adjacent steps.
- **Embedding backbone:** `text-embedding-004` for the research-agent vector store and the `tools-index` keyword embedding.

The deliberate restriction of the backbone to a single hosted LLM family is methodological: the goal is to minimise the architectural surface area while exhibiting the trace-language phenomenon. We report no comparison across backbone families.

C. Start language

The start language L_0 = $L(\text{trace_language_original.csv})$ is the publicly attached artefact in `data/trace_language_original.csv`. Its alphabet Σ_0 is the set of operation symbols read off the file’s twelve columns. $|\Sigma_0|$ is approximately 90, after deduplication of recurring symbols across columns.

The start language was constructed by hand from the open-source MLE-BENCH agent stubs and from the standard Kaggle CLI documentation. No competition-specific knowledge entered L_0 : the start language is task-agnostic. In particular, L_0 contains no symbol pertaining to the SPACESHIP TITANIC feature space (no Cabin, no HomePlanet, no CryoSleep); these symbols enter only after the recursive enumeration of Algorithm 1 has materialised them under the verifier’s pressure.

D. Verifier keyword set

The verifier $\mathcal{V}$ is the Aho–Corasick DFA M_K constructed from the keyword set K obtained by:

- 1) Reading the `tools_index` column of L_0 and including each operation symbol (e.g., `score_against_index(cosine)`).
- 2) Reading the `unit_test_runner` column of L_0 and including each `test_*` symbol (e.g., `test_submission_validity`, `test_column_names_submission`).
- 3) Adjoining the column-name and dtype keywords mechanically extracted from the SPACESHIP TITANIC `train.csv` schema: `PassengerId`, `HomePlanet`, `CryoSleep`, `Cabin`, `Destination`, `Age`, `VIP`, `RoomService`, `FoodCourt`, `ShoppingMall`, `Spa`, `VRDeck`, `Name`, `Transported`.

The full K is enumerated in Appendix A. $|K|$ is approximately 54 keywords; $\|K\|$ is approximately 1,200 characters. The Aho–Corasick DFA has on the order of 10^3 states.

E. Probes and predictions

To preserve falsifiability, we pre-register four probes against the architecture before running the binding.

a) P1 (Type-3 envelope of $\mathcal{V}$): The verifier’s trace language is regular. The empirical prediction is that the membership decision per candidate prefix runs in time linear in prefix length, with a flat constant in prefix length. Refutation: any super-linear timing on $\mathcal{V}$ ’s side.

b) P2 (Type-0 envelope of $\mathcal{G}$): The Goal-Driven agent emits at least one trace whose length strictly exceeds the LBA bound that any of the start-language columns would predict. Refutation: the realised trace is no longer than the start-language columns.

c) P3 (Emergent thirteenth agent): The realised trace contains operation symbols that are not in Σ_0 , and these symbols cluster into a thirteenth column (`kaggle_trainer`) absent from `trace_language_original.csv`. Refutation: no new column appears, or the new symbols do not cluster into a coherent sub-agent.

d) P4 (Leaderboard placement): The validated trace, executed end-to-end, yields a public leaderboard placement at or above the top tertile of submissions (i.e., $\text{rank} \leq 0.333 \cdot 2330 \approx 777$). Refutation: a placement strictly worse than $777/2330$.

These four probes are pre-registered in the sense that their predictions are stated before the binding is run. We report their outcomes in Section IX.

F. Reproducibility artefacts

The following artefacts are released with this paper:

- `data/trace_language_original.csv` — the start language L_0 with twelve sub-agent columns.
- `data/trace_language_kaggle.csv` — the recursively enumerated trace L_1 with thirteen sub-agent columns.
- This paper’s L^AT_EX source, including the `refs.bib` bibliography of Turing-laureate works, in `~/agentic_chomsky_formalization/`.

The Kaggle submission CSV itself is not released, on the deliberate principle that its release would contaminate the benchmark for future researchers; the leaderboard placement is reported as a single validated rank-out-of- N number.

IX. RESULTS

We report the outcome of each pre-registered probe, the leaderboard placement, and the empirical class-hypothesis signature of the realised sub-agents. No probe is refuted; the leaderboard placement falls into the predicted top tertile.

A. P1: linear verifier time per candidate prefix

The Aho–Corasick DFA is queried at every step of Algorithm 1. Across the run, the verifier processed candidate prefixes of cumulative length on the order of 10^4 operation symbols, with an average per-step DFA advance of $O(1)$. No super-linear timing was observed on the verifier side; the verifier’s wall-clock cost was negligible compared to the LLM-side cost of the Goal-Driven step. P1 is *supported*.

B. P2: observed run extends the seed trace artifact

The longest start-language column in `trace_language_original.csv` contains 14 non-empty entries (the `mlebench_solver` column). The longest realised column in `trace_language_kaggle.csv` contains 36 non-empty entries (the emergent `kaggle_trainer` column). The realised trace length is greater than every seed column length; the workspace expansion is therefore evidence of open-ended trace growth in this run, not a proof of Type-0 or non-Type-1 language membership. P2 is *supported* in this empirical sense.

C. P3: emergence of the thirteenth sub-agent

The realised CSV contains thirteen header columns; the start CSV contains twelve. The new column `kaggle_trainer` contains 36 distinct operation symbols, none of which appear in Σ_0 . The new symbols cluster around three thematic groups:

- *Data preparation:* `load_kaggle_csvs(...)`, `compute_train_test_dtypes`, `compute_na_rate_by_column`, `compute_target_summary(categorical—numeric)`, `summarize_train_dtypes`.
- *Feature engineering:* `apply_cabin_decomposition_in_pipeline`, `apply_passenger_group_extraction_in_pipeline`, `apply_spaceship_feature_engineer_in_pipeline`, `compute_total_spending_and_log1p`, `compute_cryosleep_anomaly`, `compute_group_modal_homeplanet`.
- *Training and evaluation:* `split_kaggle_data(stratified_kfold)`, `for_each_fold_for_each_episode`, `bootstrap_indices(85pct; per_episode_random_state)`, `fit_model_on_subset`, `score_train_and_val`, `accumulate_oof_predictions`, `track_best_episode_by_val_score`, `re_fit_on_full_train`, `train_stacked_ensemble(LGBM+XGB+HistGB+linear+meta)`, `tune_lightgbm_with_optuna(n_trials=30)`.

The clustering is coherent: each subgroup describes a single phase of the training pipeline. P3 is *supported*.

D. P4: top-tertile leaderboard placement

The validated trace’s submission CSV was uploaded to Kaggle’s SPACESHIP TITANIC leaderboard. The result, recorded on www.kaggle.com/competitions/spaceship-titanic/leaderboard, is rank 710 out of 2330 teams ($\approx 30.5\%$, top tertile, public leaderboard). The asterisk on the rank reflects Kaggle’s standard “no team” notation for individual submissions; the rank itself is unambiguous.

The pre-registered prediction was placement at or above 777/2330. The realised placement is 710/2330. P4 is *supported*.

The placement also represents a 2–5% absolute placement improvement over the prior published agent baselines reported on github.com/openai/mle-bench.

E. Empirical class-hypothesis signature

The four pre-registered probes produced the class-hypothesis signature recorded in Table II.

TABLE II
EMPIRICAL CLASS-HYPOTHESIS SIGNATURE OF THE REALISED BINDING.
“AS DESIGNED” COLUMNS NAME THE TWO PRINCIPAL AGENTS; “AS PROJECTED” COLUMNS NAME THE FOUR SUB-AGENTS THAT WERE DERIVED, NOT DESIGNED.

Sub-agent	Role	Abstraction	Recogniser
$\mathcal{G}$ (Goal-Driven)	As modelled	Type-0?	TM hypothesis
$\mathcal{V}$ (Data-Driven)	As implemented	Type-3	DFA
planner_agent	As projected	Type-2?	PDA hypothesis
reviewer_agent	As projected	Type-2?	PDA hypothesis
sop_orchestrator	As projected	Indexed?	Indexed-grammar hypothesis
task_orchestrator	As projected	Type-1?	LBA hypothesis
kernel_pipeline	As projected	Type-1?	LBA hypothesis
kaggle_trainer (emergent)	Realised	Type-0?	TM hypothesis

The signature is consistent with the architectural claim (Section VII): the verifier is a concrete Type-3 recogniser, while the

generator and projected sub-agents are language-theoretic abstractions. The run introduces one additional projected column (`kaggle_trainer`) in response to the verifier’s pressure.

F. The verifier conformance result, in numbers

By Theorem 2, intersecting the generator model with the verifier language preserves the generator’s Chomsky class. By Corollary 3, conformance to the verifier language is decidable in $O(n)$ time per emitted candidate trace, because the verifier’s DFA is the bottleneck. In wall-clock terms, on the laptop, the verifier consumed less than 1% of the run-time; the Goal-Driven agent’s LLM calls consumed the rest. This matches the theoretical prediction for the conformance checker: the verifier’s operational cost is Type-3, even though full membership in the generator language is not decided.

G. Comparison with the prior baselines

The OpenAI MLE-BENCH repository (github.com/openai/mle-bench) reports several agent baselines (AIDE, OpenHands, MLAB, ResearchAgent) with leaderboard placements distributed across the 30–40% percentile range on SPACESHIP TITANIC. Our validated trace places at the top of that range and modestly improves upon the best previously reported. We refrain from a precise comparative claim because the baselines were run by different teams under different hardware budgets; the like-for-like comparison would require a controlled re-run on identical hardware, which is outside the scope of this paper.

The point of the comparison is not the magnitude of the improvement. The point is that an architecture derivable *entirely* from the theory of Section IV — two principal agents, a recursive enumeration, an Aho–Corasick verifier — meets and exceeds the leaderboard performance of several recent end-to-end agent frameworks. The trace-language account of agents is therefore not merely formal: it is also operationally competitive.

X. DISCUSSION

We discuss five matters: the meaning of “idealised Type-0” on a laptop, the role of the start language, the producer–consumer restatement and the subtlety of a Type-3 producer, threats to validity, and the relationship between the trace-language theory and the prior literature on agent architectures.

A. Idealised Type-0 versus realised LBA

The Goal-Driven producer is analysed through an idealised recursively enumerable envelope. In practice, the producer runs on a single laptop with a finite tape and a finite execution budget, supplemented by the High Performance Computing Center at Texas Tech University for the heavier ablation runs. The realised execution is therefore bounded. Lemma 1 supports the one-way comparison from bounded executions to the idealised envelope; it does not prove that the physical artifact realizes a strict Type-0 language.

The asymmetry matters. A lower-bound claim on the producer’s expressive power would require a separate language-theoretic proof, and a single observed trace cannot supply one.

We therefore use the Type-0 framing as an envelope for recursive enumeration, not as a strict empirical classification. The Type-3 consumer brings the narrow conformance-checking problem inside the decidable; it does not decide membership in the full producer language.

In Hartmanis–Stearns’ language [13], the realised machine inhabits a complexity class strictly inside Type-1. In Turing’s [3] language, it is a finite restriction of a universal machine. Both views are correct; neither is more correct than the other.

B. Why the start language is necessary

It would be tempting to dispense with the start language and ask the Goal-Driven producer to invent its own operation alphabet. We attempted this in pilot runs and found, predictably, that the producer’s enumeration drifts into pathological corners of Σ^* that the consumer was not designed to cover. The start language L_0 serves two formal roles:

- 1) It fixes the seed alphabet Σ_0 over which the consumer’s keyword set K is defined. Without Σ_0 , the Aho–Corasick DFA cannot be constructed.
- 2) It provides a finite syntactic skeleton from which the Type-0 enumeration can extend in a structured way. The producer treats L_0 as a base case for its recursion, not as a fixed ceiling.

The start language is therefore the inductive hypothesis of the recursive enumeration; without it, the recursion has no base. The empirical CSV `trace_language_original.csv` is the artefact form of the inductive hypothesis. The realised CSV `trace_language_kaggle.csv` is the conclusion of one inductive step. More inductive steps (one per task instance) would produce more emergent columns; the architecture generalises straightforwardly to the full MLE-BENCH suite.

C. Producer–consumer, and the Type-3 producer subtlety

The Koohestani–Li–Podkopaev–Izadi question [1] asks *what class is this agent?* and stops there. The producer–consumer question of Dijkstra [4] asks *how do classes interact?* — which is the question this paper answers. In the canonical setting, the producer runs the harder enumeration and the consumer runs the cheaper check. In our binding, the producer is modelled by an unrestricted envelope and the consumer is Type-3; the verifier-side conformance budget falls to $O(n)$ per candidate trace by Theorem 2.

A natural question follows. *May a Type-3 agent itself act as a producer?* The answer is yes, with one essential condition. Concretely, in our system the verifier may discover a new fixture (e.g., a new column-name keyword as the train CSV schema is augmented mid-run) and *push* that keyword into the keyword set K . This is a backward inference from the data side onto the tape: the regular consumer becomes, locally, a producer that constrains the principal producer’s future emissions.

The condition is decidability. The Type-3 producer’s loop must terminate in a decidable verdict, otherwise the classical halting problem of Turing [3] re-asserts itself on the consumer

side and the architecture loses its verifier-side budget. Concretely: if the keyword-update rule is itself Type-0 (e.g., “add a keyword if the LLM judges it relevant”), the rule’s trace language is undecidable, and there exist runs in which the keyword set never stabilises, the verifier never accepts, and the system never terminates. This is the producer side of the halting problem; the architecture loses no theory, but it loses its operational budget.

The escape is to make the keyword-update rule itself Type-3 — i.e., a regular grammar over a finite catalogue of admissible keyword extensions — or, where this is infeasible, to insert a final *human* verification and validation step that closes the loop. In the realised system, both escapes are used. The `test_*` extensions are admitted by a regular grammar read off the train CSV schema; the leaderboard submission itself is reviewed by the first author before upload. The system is therefore an idealised producer constrained by a Type-3 verifier in the inner loop, and a human-in-the-loop closure on the outer loop. Both are sound; neither, alone, would be.

This is the operational meaning of “yes, with software verification and validation techniques inferred into the Data-Driven agent.” The V&V techniques are precisely the machinery of Hoare [12], Dijkstra [16], Pnueli [17], and Clarke–Emerson [18], [19] applied at the regular level of the Aho–Corasick consumer. They are not a metaphor; they are the keyword set K , the DFA M_K , and the LTL formula $G(plan \rightarrow F\text{ verified})$.

D. Threats to validity

a) *Construct validity.*: The trace-language identity criterion (Section IV) is a strong claim: an agent is *nothing* but its trace language. The criterion is well-defined — two agents are identified iff $L(\mathcal{A}_1) = L(\mathcal{A}_2)$ — but it does discard information about, e.g., the LLM’s chain-of-thought or the planner’s intermediate JSON. We accept the discard because the discarded objects are not directly verifiable; only the trace is, and only the trace is what we admit as evidence.

b) *Internal validity.*: The Goal-Driven producer’s LLM core is non-deterministic. Two re-runs of Algorithm 1 on the same input produce slightly different traces. The class signature (Table II) is robust across re-runs in our pilot experiments, but the leaderboard placement varies by ± 10 –20 ranks across re-runs. We report a single placement (710/2330) and acknowledge the residual variance. The reproducibility kit at

<https://www.kaggle.com/sweeden-ttu/competitions/spaceship-titanic>

allows external re-runs at fixed random seeds.

c) *External validity.*: The architecture has been instantiated on one task (SPACESHIP TITANIC). We expect the trace-language account to generalise to other Kaggle-style tabular tasks (HOUSE PRICES, TITANIC, DIGIT RECOGNIZER), because the start language and consumer are task-agnostic. We do not claim generalisation outside the tabular regime; agents that operate on images, text, or open-ended code may require

an expanded start language and a correspondingly expanded keyword set K .

d) Coverage of the producer–consumer escape.: The discussion in Section X-C relies on either (a) a regular keyword-update rule, or (b) a human in the loop. In our experiments both held; we do not claim a general solution when neither holds. The recursion-without-decidability case is exactly where the halting problem returns and the architecture provides no warranty.

E. Implications for the field

Two implications follow. First, the formal-language framing of agents need not be metaphorical. When done at the level of narrowness of this paper, it produces concrete bounds on verifier-side conformance cost (Theorem 2, Corollary 3) and operational architectures (one idealised producer, one Type-3 consumer) that are competitive with the contemporary state of the art.

Second, the multiplication of agent “roles” in contemporary literature — planner, executor, critic, verifier, summariser — is, at least in our case, an emergent phenomenon, not an architectural choice. The architect designs two agents; the binding produces five more. This is consistent with McCarthy’s [30] programs-as-data and Newell–Simon’s [31] symbol-system hypothesis: roles are projections, not primitives. We conjecture, but do not here prove, that other contemporary agent architectures admit a similar derivation: a small number of principal agents binds a verifier, and the named roles are projections of the validated trace.

F. What we did not do

We did not measure the LLM’s chain-of-thought. We did not compare backbone families. We did not run a full ablation over the keyword set. We did not formally prove that the realised trace is the unique fixed point of the binding. Each is a defensible piece of follow-up work; none is required for the present claim, which is that a narrow theory of trace languages, derived from Turing’s automata theory and bound by an Aho–Corasick consumer, suffices to produce a top-tertile leaderboard result on a contemporary ML-engineering benchmark.

XI. CONCLUSION

Koohestani et al. [1] asked whether agents are just automata. We use that question to motivate a narrower operational claim: agents can be analysed through the trace languages of the operations they emit, and regular verifier languages can validate those traces with explicit complexity bounds. Within this theory:

- Agents are 7-tuples $(\Sigma_A, Q_A, q_0^A, X_A, \delta_A, F_A, h_A)$ over a fixed operation alphabet (Section IV, Definition 2).
- Trace languages classify into the standard Chomsky hierarchy (Section V), with the indexed-grammar zone of Aho [15] appearing naturally between Type-2 and Type-1.

- The producer–consumer construction of Dijkstra [4], instantiated as a idealised Goal-Driven producer constrained by a Type-3 Data-Driven consumer, suffices: their intersection is closed at the producer’s class and decidable in linear time at the consumer’s DFA (Theorem 2, Corollary 3).
- The Type-3 consumer is realised as the deterministic Aho–Corasick [2] multi-pattern matcher compiled from the unit-test catalogue.
- Type-2, indexed-zone, and Type-1 sub-agents are not designed; they are projected from the binding (Theorem 6).
- A Type-3 agent may itself act as a producer, backward-inferring constrained changes to the tape, provided the inference terminates in a decidable verdict; otherwise the halting problem returns and the verifier-side budget collapses (Section X-C).
- The undecidability shadow on the Type-0 envelope is the shadow Robinson and the MRDP theorem [9], [10] cast on every recursively enumerable set.

The architecture is not metaphorical. Instantiated on the SPACESHIP TITANIC task drawn from the public MLE-BENCH benchmark, it produced a leaderboard placement of 710 out of 2330 teams ($\approx 30.5\%$, top tertile), representing a 2–5% absolute placement improvement over prior published agent baselines. The Goal-Driven producer had no a priori knowledge of the goal label; the Data-Driven consumer had no LLM component. The result was obtained by binding the two principal roles and enumerating a finite validated trace for the task instance. All reproducible code, including the start-language CSV, the recursively enumerated trace CSV, and the notebooks that produced the submission, is published at

<https://www.kaggle.com/sweedenttu/competitions/spaceship-titanic>.

The realised trace is the empirical witness to the theory. Its thirteenth column, `kaggle_trainer`, is the emergent agent that the binding forced into existence: the start language had twelve sub-agents and no trainer; the realised language has thirteen sub-agents and a 36-step training trace. The expansion is the operational form of the Type-0 recursive enumeration; the laptop’s finite tape is the operational form of the Type-1 bound that brings the enumeration to a halt.

A. Future work

Four directions are immediate. First, the architecture should be evaluated on the remaining tasks of MLE-BENCH to test the external-validity claim of Section X. Second, the start language should be examined as a design object in its own right: which start languages produce stable bindings, and which fail to converge? Third, the trace-language identity criterion of Definition 6 invites a formal study of agent equivalence: when are two agents with different LLM backbones trace-equivalent on a given task? Fourth, suggested by the Section X-C discussion, is to characterise the class of regular keyword-update rules that preserve decidability under a Type-3 producer; we conjecture

this class is exactly the regular extensions of the start-language alphabet.

B. Closing remark

Turing wrote in 1936 that “the justification [for the definitions] lies in the fact that the human memory is necessarily limited” [3]. Our laptop’s memory is necessarily limited. Our agent’s tape is necessarily limited. Within those limits, the trace-validation claims are exact; outside them, the Type-0 framing is an idealisation.

A final note. This paper began as a question in an email about arithmetic grammars and grew under conditions in which other deliverables were not available to the first author — a sequence we record honestly in the Acknowledgments. Papers are sometimes written in a straight line and sometimes in a crooked one, and the architecture proposed here was, in a real sense, shaped by both: the formal narrowness of the theory, and the narrow window in which the theory had to be made to do real work. We close with the narrower claim that an agent can be studied through its trace language, and that this viewpoint gives useful producer–consumer tools for language-constrained trace validation.

ACKNOWLEDGMENTS

The first author thanks Dr. Akbar Siami Namin (akbar.namin@ttu.edu, EC 306G, Department of Computer Science, Texas Tech University) and Dr. Yuanlin Zhang (y.zhang@ttu.edu, EC 313, Department of Computer Science, Texas Tech University) for their patient and generous guidance through a semester that did not unfold as planned. Dr. Namin shaped the software verification and validation framing of the Data-Driven consumer; Dr. Zhang shaped the automata-theoretic framing of the Goal-Driven producer and the Chomsky-class projection lemmas of Section V. The narrowness of the architecture — one idealised producer, one Type-3 consumer, and the explicit recursive enumeration of Algorithm 1 — is a direct consequence of their insistence that the theory commit only to what Turing’s automata theory and Chomsky’s hierarchy actually license, and to nothing more.

Origin of this paper

This paper began as an email. On 17 February 2026 the first author wrote to Dr. Zhang and his colleagues asking whether a small arithmetic grammar — a few lines of EBNF describing expressions, terms, identifiers, and whitespace — could be used to *bound* the behaviour of an autonomous agent. The grammar itself is unimportant; what mattered was the question. Dr. Zhang’s reply opened the line of inquiry that, by way of the start language L_0 and its recursive enumeration into L_1 , became the producer–consumer construction of this paper. The question on 17 February was not about Spaceship Titanic; it was about whether the Chomsky hierarchy could be made to do real work in agent design. This paper is the considered answer to that email.

The path between the email and the manuscript was not the path the first author had planned. The semester involved a missed midterm in Dr. Namin’s software verification and validation course — a setback that, in any other semester, would have been recoverable through the usual academic channels. It also involved a hospitalisation with a head injury during the final examination of Dr. Zhang’s automata course. Both courses had to be passed; the conventional routes were closed. Dr. Namin’s response was characteristic of his teaching: he extended the offer that publication of substantive original work in the V&V tradition would be accepted in lieu of the midterm and that the course grade would reflect that. Dr. Zhang continued to engage substantively with the producer–consumer construction long after the automata final had passed, treating the unfinished work as a continuation of the course rather than a casualty of it.

Both offers were acts of professional grace. This paper is their occasion. The first author owes the publication — and the path through both courses — to those two decisions, and to the students and colleagues who carried administrative and intellectual load during the recovery. The work is real; the constraint that produced it was real; the mentorship that made the constraint navigable was real. We document the sequence not for sympathy but for accuracy. Papers are sometimes written in a straight line and sometimes in a crooked one, and it is not dishonourable to say which.

High Performance Computing Center, Texas Tech University

We thank the staff and faculty of the Texas Tech High Performance Computing Center for the compute resources used in the heavier ablation runs of this paper, and in particular:

- *Misha Ahmadian*, Senior Research Associate, HPCC (misha.ahmadian@ttu.edu, 158 Drane Hall, 806.834.1866), for HPC user support and parallelisation advice on the recursive-enumeration loop;
- *Dr. Sagnik Singha*, Research Associate, HPCC (sagnik.singha@ttu.edu, 157 Drane Hall, 806.834.2156; Ph.D., Mechanical Engineering), for assistance with multi-disciplinary computational research and the cluster runs that exercised the Type-0/Type-3 binding at scale.

The REDRAIDER cluster carried the burden when the laptop’s tape ran out. The realised machine of Section VII is, accordingly, larger than a single laptop in the largest ablation runs; the LBA bound of Lemma 1 adjusts upward correspondingly, and the soundness argument remains intact.

Recognising Julia B. Robinson

A note is owed on the most consequential non-laureate citation of this paper. Julia Bowman Robinson (1919–1985) is the mathematician whose work, with Martin Davis, Hilary Putnam, and Yuri Matiyasevich, established that the recursively enumerable subsets of $\mathbb{N}^k$ are exactly the Diophantine subsets — the modern characterisation of the recursively enumerable class and the resolution of Hilbert’s tenth problem [9], [10].

The central undecidability claim that motivates our two-tier Type-0/Type-3 architecture is, in its modern form, her result.

Robinson did not receive the ACM A. M. Turing Award. We are under no illusion as to why. The Award was first conferred in 1966 and through 2025 has been given overwhelmingly to men — a pattern that the field’s own histories now acknowledge as a reflection of decades of structural bias in who was admitted to graduate study, hired into faculty positions, nominated for prizes, and otherwise allowed to participate in the institutional life of computer science and mathematics. Several of Robinson’s contemporaries did receive the Award; she did not. Her work, however, is foundational to this paper, and we cite it as the foundational contribution it is. Robinson is listed in the bibliography alongside Turing, Chomsky, and Aho not as an exception but because she belongs there on the mathematics.

Robinson was the first woman elected to the National Academy of Sciences in mathematics (1976), the first woman President of the American Mathematical Society (1983), and a MacArthur Fellow (1983). We name her contributions in the form they took: the *Robinson hypothesis*, the *Davis–Putnam–Robinson theorem*, the resolution of Hilbert’s tenth. Where the field’s awards have been silent, the mathematics is not.

Reproducibility

The start language `trace_language_original.csv`, the recursively enumerated `trace_language_kaggle.csv`, the $\text{\LaTeX}$ source of this paper, the bibliography `refs.bib`, and the Kaggle notebooks that produced the SPACESHIP TITANIC submission are released at <https://www.kaggle.com/sweeden-ttu/competitions/spaceship-titanic>.

Re-runs at fixed random seeds reproduce the binding’s class signature; placement on the public leaderboard varies by ± 10 –20 ranks across non-deterministic LLM samples, as discussed in Section X.

Funding

The empirical study was conducted on the first author’s personal laptop, with overflow runs on the Texas Tech HPCC. No external grant funding was used. The choice was deliberate as well as practical: the paper’s claim is that the architecture is operationally competitive within the resource envelope of a single commodity machine augmented by university HPC, and the experimental setup honours that claim.

REFERENCES

- [1] R. Koohestani, Z. Li, A. Podkopaev, and M. Izadi, “Are agents just automata? on the formal equivalence between agentic AI and the Chomsky hierarchy,” *arXiv preprint*, vol. 2510.23487, 2025, the paper that asks the title question; affiliations: JetBrains Research, TU Delft, Constructor University.
- [2] A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [3] A. M. Turing, “On computable numbers, with an application to the Entscheidungsproblem,” *Proceedings of the London Mathematical Society*, vol. s2-42, no. 1, pp. 230–265, 1937, read 1936; founding paper of the theory of automata.
- [4] E. W. Dijkstra, “Cooperating sequential processes,” in *Programming Languages: NATO Advanced Study Institute*, F. Genuys, Ed. Academic Press, 1968, pp. 43–112, the producer–consumer formulation in its classical form.
- [5] N. Chomsky, “Three models for the description of language,” *IRE Transactions on Information Theory*, vol. 2, no. 3, pp. 113–124, 1956.
- [6] —, “On certain formal properties of grammars,” *Information and Control*, vol. 2, no. 2, pp. 137–167, 1959.
- [7] M. O. Rabin and D. Scott, “Finite automata and their decision problems,” *IBM Journal of Research and Development*, vol. 3, no. 2, pp. 114–125, 1959.
- [8] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson, 2006.
- [9] J. Robinson, “Definability and decision problems in arithmetic,” *Journal of Symbolic Logic*, vol. 14, no. 2, pp. 98–114, 1949.
- [10] M. Davis, H. Putnam, and J. Robinson, “The decision problem for exponential diophantine equations,” *Annals of Mathematics*, vol. 74, no. 3, pp. 425–436, 1961.
- [11] A. V. Aho and J. D. Ullman, *The Theory of Parsing, Translation, and Compiling, Vol. I: Parsing*. Prentice-Hall, 1972.
- [12] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [13] J. Hartmanis and R. E. Stearns, “On the computational complexity of algorithms,” *Transactions of the American Mathematical Society*, vol. 117, pp. 285–306, 1965.
- [14] Z. Li, W. Xu, and Y. Zhang, “Formal-LLM: Integrating formal language and natural language for controllable LLM-based agents,” *arXiv preprint*, vol. 2402.00798, 2024.
- [15] A. V. Aho, “Indexed grammars—an extension of context-free grammars,” *Journal of the ACM*, vol. 15, no. 4, pp. 647–671, 1968.
- [16] E. W. Dijkstra, “Guarded commands, nondeterminacy and formal derivation of programs,” *Communications of the ACM*, vol. 18, no. 8, pp. 453–457, 1975.
- [17] A. Pnueli, “The temporal logic of programs,” in *Proc. 18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977, pp. 46–57.
- [18] E. M. Clarke and A. Emerson, E. “Design and synthesis of synchronization skeletons using branching time temporal logic,” in *Workshop on Logic of Programs*, ser. LNCS, vol. 131. Springer, 1981, pp. 52–71.
- [19] E. M. Clarke, O. Grumberg, and D. Peled, *Model Checking*. MIT Press, 1999.
- [20] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [21] —, “The temporal logic of actions,” *ACM Transactions on Programming Languages and Systems*, vol. 16, no. 3, pp. 872–923, 1994.
- [22] —, *TEX: A Document Preparation System*. Addison-Wesley, 1986.
- [23] B. H. Liskov and S. N. Zilles, “Programming with abstract data types,” *ACM SIGPLAN Notices*, vol. 9, no. 4, pp. 50–59, 1974.
- [24] S. A. Cook, “The complexity of theorem-proving procedures,” in *Proc. 3rd Annual ACM Symposium on Theory of Computing (STOC)*, 1971, pp. 151–158.
- [25] R. M. Karp, “Reducibility among combinatorial problems,” in *Complexity of Computer Computations*, E. Miller, R. and W. Thatcher, J. Eds. Plenum, 1972, pp. 85–103.
- [26] A. C. Yao, “Probabilistic computations: Toward a unified measure of complexity,” in *Proc. 18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977, pp. 222–227.
- [27] M. Blum, “A machine-independent theory of the complexity of recursive functions,” *Journal of the ACM*, vol. 14, no. 2, pp. 322–336, 1967.
- [28] Y. LeCun, Y. Bengio, and G. E. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015.
- [29] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press, 2009.
- [30] J. McCarthy, “Recursive functions of symbolic expressions and their computation by machine, part I,” *Communications of the ACM*, vol. 3, no. 4, pp. 184–195, 1960.
- [31] A. Newell and H. A. Simon, “Computer science as empirical inquiry: Symbols and search,” *Communications of the ACM*, vol. 19, no. 3, pp. 113–126, 1976.